ECOLE
POLYTECHNIQUE
DE BRUXELLES

DATA-DRIVEN
Séance 2

25/02/2026

EXO 2 PCA

POD INTRO

Apply **Proper Orthogonal Decomposition (POD)** to a dataset of:

$$400\ \textit{face images}\ (64 \times 64\ \textit{pixels})$$

in order to:

- identify the **dominant spatial patterns** in facial images
- construct a **reduced-order representation** of the dataset
- reconstruct faces using only a **limited number of modes**

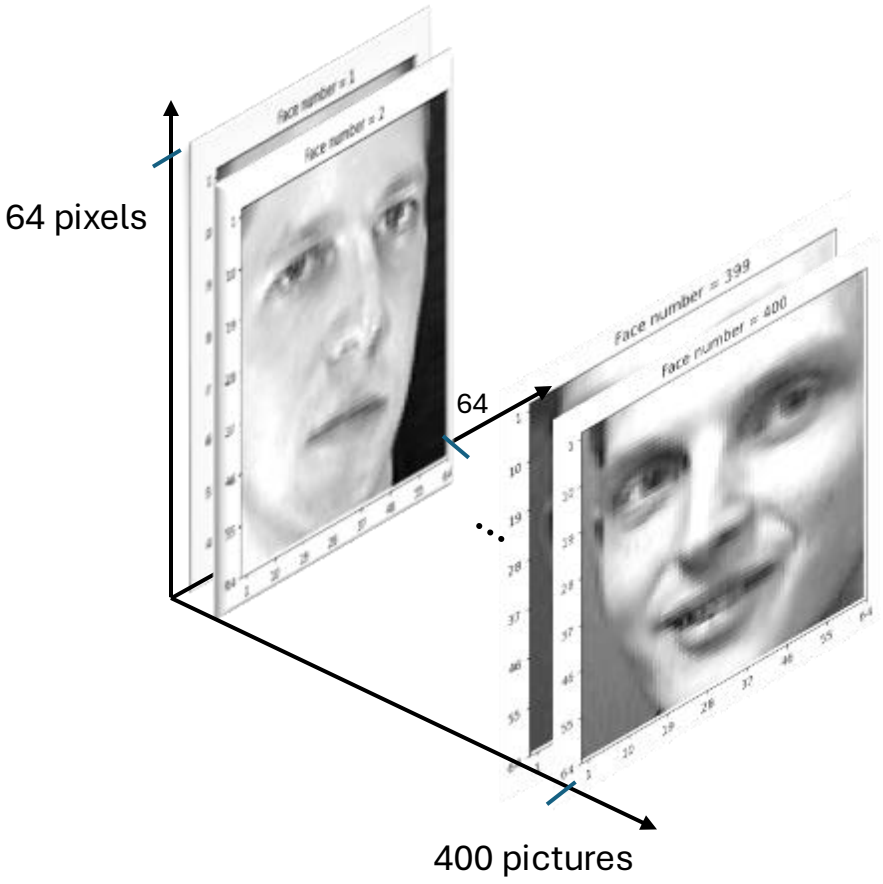
Each face image $\mathbf{x} \in \mathbb{R}^{4096}$ can be approximated as:

$$\mathbf{x} \approx \mu(\mathbf{x}) + \sum_{i=1}^{q} a_i \boldsymbol{\phi}_i$$

where:

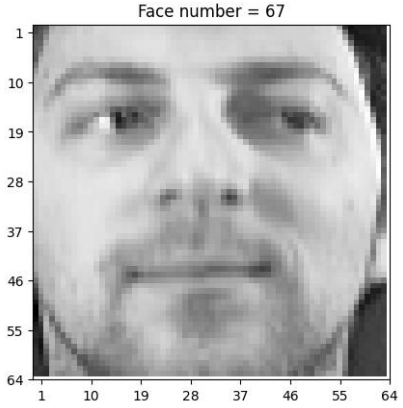
- $\mu(\mathbf{x})$: mean face
- $\boldsymbol{\phi}_i$: POD modes (eigenfaces)
- a_i : modal coefficients
- $q \ll 4096$

Dataset



$face_images$.shape = (400,64,64)

Grayscale: pixels range from 0 (black) to 1 (white). Let's take one of the image



$$= \begin{bmatrix} 0.24 & \cdots & 0.18 \\ \vdots & \ddots & \vdots \\ 0.21 & \cdots & 0.25 \end{bmatrix} \updownarrow 64$$

$\longleftrightarrow$ 64

Task 1: reshape $face_images$ into face_X, of shape (64*64 , 400), such that each column contains a full face

$$\begin{bmatrix} & \cdots & \\ \vdots & \ddots & \vdots \\ & \cdots & \end{bmatrix} \updownarrow 64*64$$

$\longleftrightarrow$ 400

Hint: use the np.reshape function

- To be able to apply POD, we need a matrix → Please reshape the **tensor** face_images as a **matrix, EXEMPLE :**

We have **3 faces**, each of size **2×2 pixels**

$$\text{Dataset shape} = (3, 2, 2)$$

Face 1, Face 2, Face 3

$$F_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \qquad F_2 = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \qquad F_3 = \begin{bmatrix} 10 & 11 \\ 12 & 13 \end{bmatrix}$$

Face 1

$$X = \begin{bmatrix} 1 & 2 & 10 \\ 2 & 3 & 11 \\ 3 & 4 & 12 \\ 4 & 5 & 13 \end{bmatrix}$$

$$X \in \mathbb{R}^{4\times 3}$$

Key difference with previous session

- Each face can be seen as a snapshot / measurement of all our pixel space.
- We want to compress along pixels = our "spatial features"
- But this would give a covariance matrix of 4096x4096, which has > 16 million entries
- Instead, we focus on the dual problem and see on how similar faces are. Now we are looking for a 400x400 covariance matrix.
- We find eigenghosts (eigenvectors)
- Thanks to SVD, the solution that we find is equivalent

In this case, we want to study how similar faces are. Each face can be seen as a “snapshot” of the 64*64 pixels

We want to see how much of our dataset we can reconstruct based on a few ”ghost” faces (eigenvectors) --> POD identifies the main directions of variation between faces (e.g. lighting or expression), ranked by the variance they capture.

We are not applying dimensionality reduction on the features (=pixels), we are applying it on our observations (=faces).

- FIRST #1 → Center matrix = remove the mean (otherwise mean itself is a strong direction= first mode becomes "the average")

$$X \in \mathbb{R}^{N \times P} \quad \rightarrow \quad X_0 = X - \bar{X}$$

Computes the **mean of each row** (i.e., the mean value of each pixel across all faces).

- STEP #2 → Compute Covariance Matrix : $K = X_0^T X_0$

It measures:
How pixels vary together.
If pixel i and pixel j increase/decrease together →their covariance is large.

So this matrix captures **relationships between variables**.

- Diagonal → variance of each face itself
- Off-diagonal → correlation between faces

STEP #3 → Eigen Decomposition

$$K\Psi = \Psi\Lambda$$

Eigenvectors : Each one is a **direction in face space** (a mode)
Eigenvalues : Measure the **amount of variance captured** by the corresponding eigenvector

STEP #4 → Sort Eigenvalues → Sort eigenvectors

$$\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \dots$$

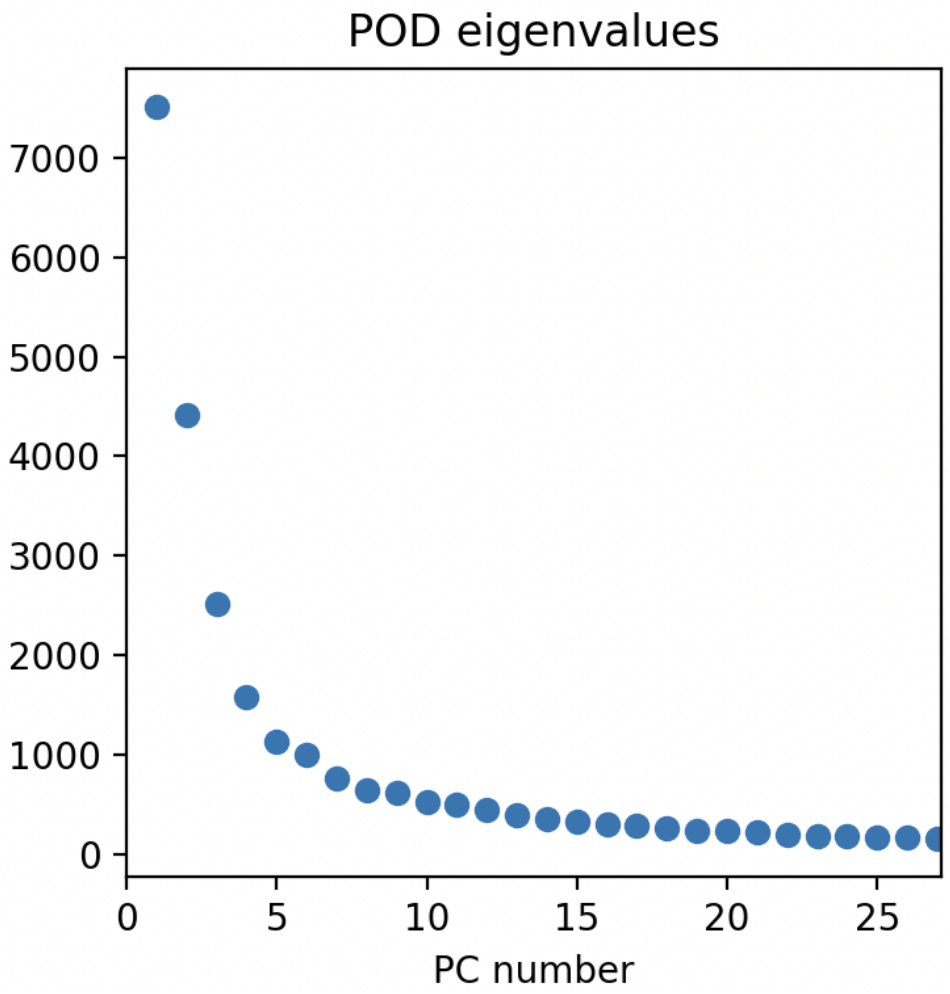
High eigenvalue → **lot of variation**
Low → low variation

STEP #5 → Compute POD Modes

$$Z = X_0\Psi$$

- Z is the **matrix of principal component (PC) scores**
- Each **column of Z** = a **POD mode expressed in pixel space** (ghost face)
- Each **row of Z** = a **pixel across all modes**

STEP #6 → Normalize



- **Steep drop-off**: First few modes capture almost all the energy.
- **Flat tail**: Later modes have tiny energy (mostly noise).

SVD INTRO

Objective

- Instead of computing the covariance matrix explicitly, we can use **Singular Value Decomposition (SVD)** to perform POD directly.
- SVD automatically gives us **POD modes** (eigenfaces) and their **importance (variance captured)**.

Why use SVD?

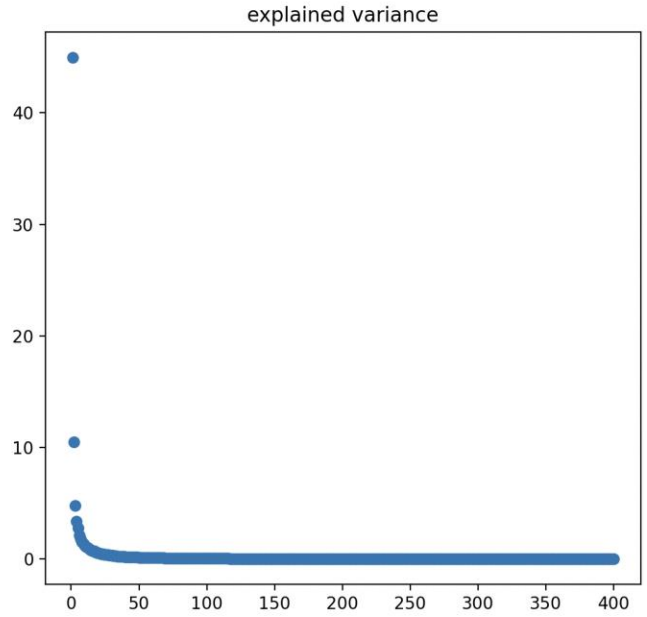
- More **numerically stable** than eigen decomposition of large covariance matrices.

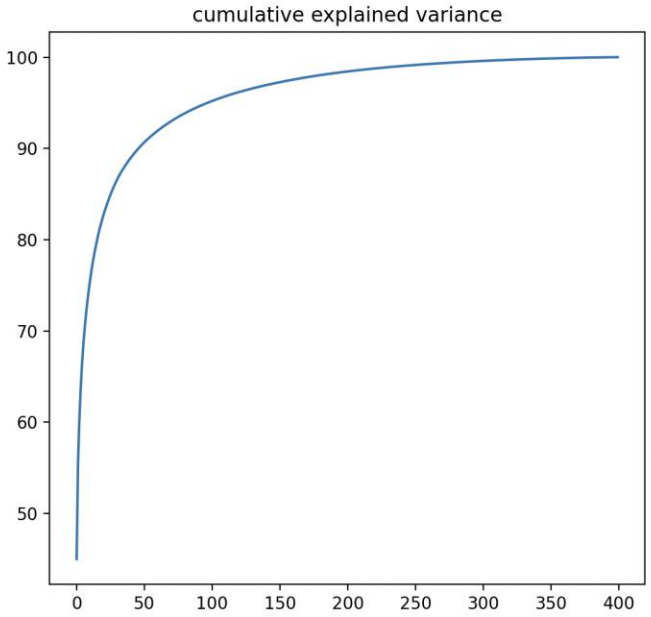
Step #1→ Center the Data
Step #2 → Compute SVD
Step #3 → Compute explained variance

Explained variance

$$\text{explained variance}_i = \frac{\sigma_i^2}{\sum_j \sigma_j^2}$$

It tells you **how much of the total variability is captured** by that mode <->
Explained variance of mode i = variance captured by this mode ÷ total variance of all modes

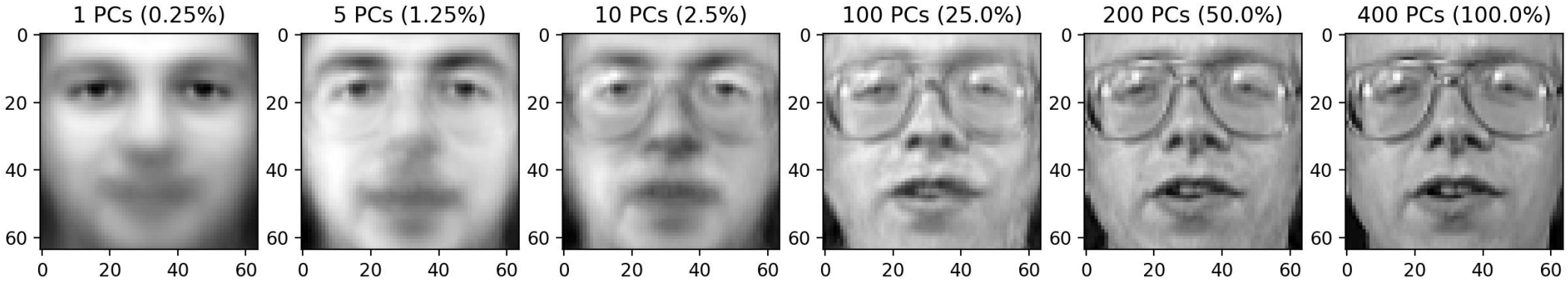




THEN we can reproduce an approx of orginal dataset (faces) with the first modes :

$$x \approx \mu + \sum_{i=1}^{q} a_i \phi_i$$

$$a_i = \phi_i^T (x - \mu)$$



CANONICAL BASIS

The canonical basis answers this question: How can I build an image from elementary building blocks?

What is it?

- In the **canonical basis**, each image is represented as a sum of **weights multiplied by basis vectors**:

$$\mathbf{x} = \sum_{i=1}^{m} w_i \mathbf{b}_i$$

$$\mathbf{b}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

- Here:
  - $\mathbf{b}_i$ = one **pixel-aligned basis vector** (1 at pixel i, 0 elsewhere)
  - w_i = weight of that pixel in the image

How it works

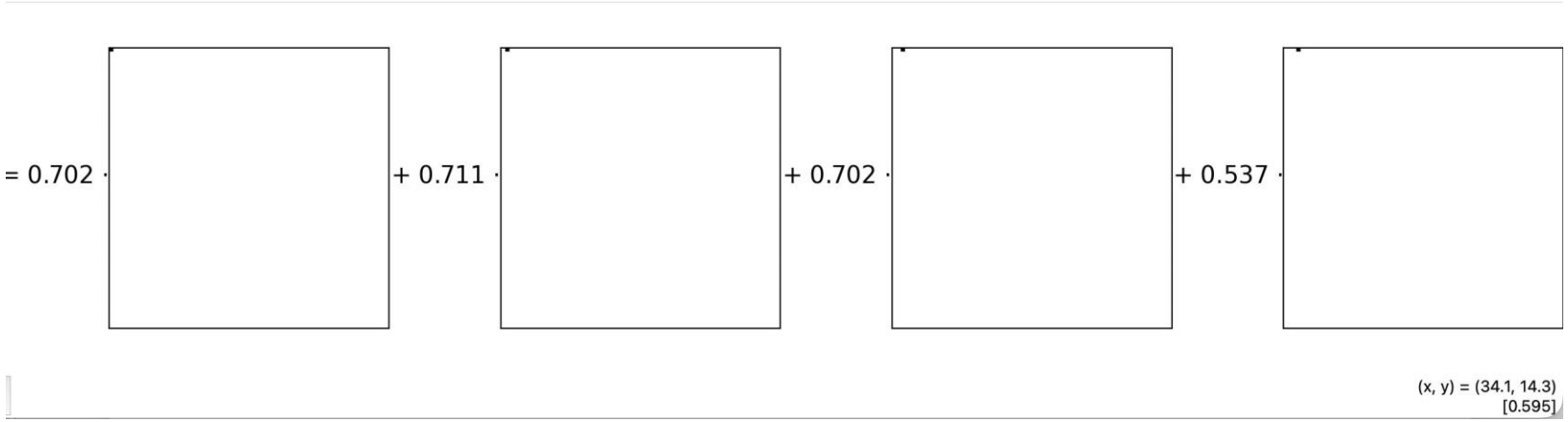
- Each image is expressed as a **linear combination of pixel-level vectors**
- All pixels are treated **independently**
- No dimensionality reduction: to reconstruct an image, you need **all pixels**

- To do:

1. Define the canonical basis, hint : identity matrix

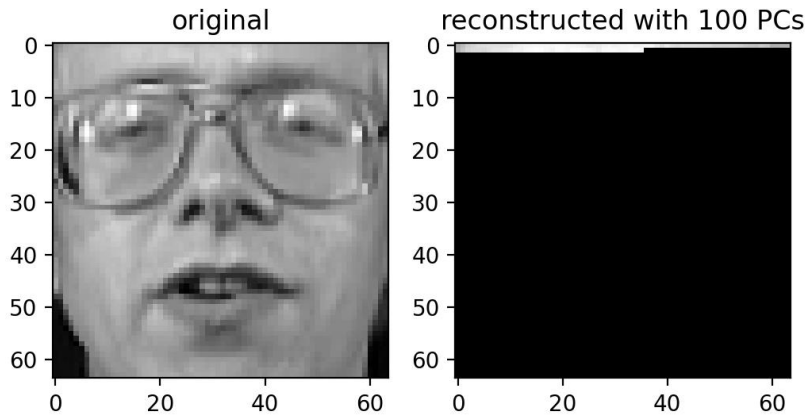
2. Obtain the weights w_i of the first 4 directions (n_basis=4),hint : think what can be the weight in our case ?

3. Show how the first 4 basis vectors look like (b_i)



weights = [0.70247936 0.7107438 0.70247936 0.5371901]

- To do: Visualizing the canonical basis decomposition



reconstruction with q=`100` canonical modes

VS

- To do: Visualizing the decomposition in POD basis

The same face reconstructed as a weighted sum of **global facial patterns** instead of individual pixels.

EXO 3 SCALING

SCALING INTRO

Goal

- Explore how **scaling affects the PCA projection**.
- Scaling changes the **relative importance of variables** in the analysis.

$$\tilde{\mathbf{X}} = (\mathbf{X} - \mathbf{C})\mathbf{D}^{-1}$$

Scaling $(\mathbf{D})$ → divide by scale factor for each variable:

ANNEXE

Each **column = one face**
Each row **= one pixel**

VERY LARGE DATASET..... BUT : You probably don't need 4096 numbers to describe a face.

Try to determine how many number we need → POD :
POD = Find the best directions to describe all faces , Instead of describing a face using **pixels**, we describe it using: "patterns that often appear in faces"

These patterns are called:
POD modes (a.k.a eigenfaces)
Each POD mode looks like:
- lighting variation
- eye distance variation
- mouth shape variation
- head orientation

face=mean face+a1ϕ1+a2ϕ2+a3ϕ3+...

Symbol	Meaning
(\mu)	average face
(\phi_i)	POD mode
(a_i)	weight (how much of this mode is present)

Compute POD using the SVD →perform POD using the SVD and compute the explained variance as a function of the modes :

Why Use SVD for POD?

Previously, we computed:

$$K = X_0^T X_0$$

and then did eigenvalue decomposition.

But:

- K is large (4096×4096 for faces)
- Computing eigenvalues of big matrices is expensive
- Proof : POD computed in 16.21 seconds
- SVD computed in 0.08 seconds

BUT : SVD decomposes it as:

$$X_0 = U\Sigma V^T$$

Where:

- Columns of V = POD modes = Φ
- Σ→ diagonal matrix of singular values σ_i
- U → (UTU=I)

$$K = V\Sigma^T\Sigma V^T$$

$$\Sigma^T\Sigma = \begin{bmatrix} \sigma_1^2 & 0 & 0 \\ 0 & \sigma_2^2 & 0 \\ 0 & 0 & \sigma_3^2 \end{bmatrix}$$

AND WE KNOW THAT : λi=σi**2

$$K = (U\Sigma V^T)^T(U\Sigma V^T)$$

$$(U\Sigma V^T)^T = V\Sigma^T U^T$$

$$K = V\Sigma^T U^T U\Sigma V^T$$

<u>Therefore, using SVD directly on X_0 automatically gives you the orthonormal basis and corresponding energies</u>